7회차 · CDC / Iceberg 프로젝트 스터디

작은 커밋이 쌓이면: Iceberg 유지보수와 내 프로젝트 리뷰

빈 커밋과 메타데이터 누적 · 4대 정리 프로시저 · 증분 시작점 보존 · 개인 설계 점검

잦은 커밋이 남긴 작은 파일과 메타데이터를 안전하게 치우고 내 파이프라인 설계를 점검합니다

오늘의 위치: 증분 다음은 유지보수와 설계 점검

6회차: 바뀐 것만 고치기

Silver changelog로 변경 감지
영향 키·날짜만 Gold MERGE
plan 선저장 · progress
후갱신

7회차 전반: 쌓인 것 치우기

10초 체크포인트와 작은 커밋
메타데이터·작은 파일 정리
순서
progress 스냅샷 보존 조건

7회차 후반 → 8회차

개인 도메인 설계·운영 리뷰
커밋·보존·정리 주기 결정
8회차 Airflow 연동·복구

빠르게 적재하고 정확히 고치는 것만큼, 쌓이는 파일과 스냅샷을 치우는 기준이 필요합니다

사고 보고: 데이터 27행, 메타데이터 3.6GB

테이블 (공통 환경 11/5)	실제 데이터	메타데이터 (snapshot · json · 크기)
bronze.orders	27행 · 파일 1개 (약 5KB)	2,952 · 2,953개 · 약 3.6GB
bronze.payments · refunds	각 28·63행 · 파일 1개	각 2,952 · 2,953개 · 약 3.6GB
silver.orders (비교)	27행 · 파일 1개	907 · 101개 · 약 110MB
연쇄 증상 (디스크 만원)	Kafka 쓰기 실패 → Connect	Rebootstrapping 로그 2.1GB

스터디 공통 환경 실측(11/5) · 눈에 띄는 Connect 로그 폭주 뒤에 Bronze 메타데이터 12.7GB(옛 cdc_events 포함)가 있었습니다

원인 ① 데이터가 없어도 커밋한다

체크포인트 10초

compose.yaml 기본 10s
Flink Iceberg 싱크는
체크포인트 완료 때 커밋

빈 커밋 허용 기본값

empty-commits 기본 10
데이터 없어도 10번째마다
빈 snapshot 커밋

나흘간 2,952번 커밋

10초 × 10 = 약 100초 간격
실측 평균 약 99초와 일치
추가된 레코드 합계는 27행

```
-- 스터디 공통 환경 실측 (bronze.orders.snapshots, 11/1 07:51 ~ 11/4 16:39 UTC)
-- 총 snapshot 2,952개 · operation 모두 'append' · 평균 간격 약 99초
-- summary['added-records'] 합계 = 27 (대부분 새 데이터 파일 없는 빈 커밋)

-- compose.yaml: CHECKPOINT_INTERVAL 기본 10s
-- Iceberg 1.10 IcebergFilesCommitter.java:
--   MAX_CONTINUOUS_EMPTY_COMMITS = "flink.max-continuous-empty-commits" (기본 10)
```

실측(11/5) + 소스 확인 · 새 이벤트가 없어도 체크포인트 10번(약 100초)마다 빈 스냅샷이 쌓였습니다

원인 ② metadata.json이 쌓이고 커진다

커밋마다 새 파일 생성

만료 전 모든 스냅샷 목록을
새 metadata.json에 기록
8KB → 2.4MB로 계속 비대

기본값은 삭제 안 함

delete-after-commit = false
previous-versions-max =
100
목록만 101개, 파일은 잔류

Bronze vs Silver 실측

Bronze: 2,953개 모두 잔류
고아 metadata.json 2,852개
Silver: 101개만 유지(110MB)

스냅샷을 만료시키지 않고 이전 metadata.json 삭제도 꺼 두면 커밋 수의 제곱에 가깝게 용량이 늡니다

Iceberg 파일 계층과 누가 무엇을 지우나

① metadata.json

스키마·파티션·스냅샷 목록
커밋마다 새 파일 생성
속성 켜면 옛 버전 자동 삭제

→

② manifest list · manifest

스냅샷별 매니페스트 목록과
데이터 파일 경로·통계(avro)
rewrite_manifests로 병합

→

③ data file (parquet)

실제 행을 담은 데이터 파일
rewrite_data_files는 합칠 뿐
옛 파일 삭제는 expire가 담당

합치기(rewrite)는 새 파일과 새 스냅샷을 만들 뿐이고, 옛 파일을 실제로 지우는 건 expire와 orphans입니다

유지보수 프로시저 네 가지와 테이블 속성

프로시저 (Spark CALL)	하는 일	주의할 점 · 관련 속성
rewrite_data_files	작은 데이터 파일을 큰 파일로 병합	새 snapshot 생성 · 옛 파일은 그대로 남음
expire_snapshots	오래된 snapshot과 참조 끊긴 파일 삭제	progress · Flink 최신 snapshot 보존 필수
remove_orphan_files	추적 밖 metadata.json · 실패 잔여물 삭제	기본 3일 · 24시간 이내 거절 · dry_run 먼저
rewrite_manifests	잘게 쪼개진 manifest avro 파일 병합	메타데이터 스캔 정리 · 새 snapshot 생성

속성 3종: delete-after-commit.enabled · previous-versions-max · flink.max-continuous-empty-commits

실험판 등장인물: lake.lesson07

구분	이름	설정과 실험 역할 (120행 · 체크섬 726,000 유지)
테이블	plain (기본값)	delete-after-commit=false (공통 Bronze와 같은 조건)
테이블	tuned (속성 적용)	delete-after-commit=true · previous-versions-max=10
테이블	inc (6회차 연결)	3번 커밋 → progress 기억 → 3번 더 커밋 후 만료 실험
5단계	grow→compact→expire→clean→trap	1행씩 120번 커밋 후 단계별 전후 파일·스냅샷 비교

유지보수는 데이터를 바꾸면 안 되므로 전 단계에서 120행과 합계 체크섬 726,000이 그대로인지 검증합니다

grow: 120번 커밋 뒤 plain vs tuned

스냅샷과 데이터 파일

둘 다 append 스냅샷 120개
데이터 파일 120개(평균 1.2KB)
데이터 객체 합계 약 976KB

metadata_log 비교

plain: 101개 (기본 상한)
tuned: 11개 (10 + 현재 1)
00111~00121번만 유지

MinIO 저장소 실물

plain: json 122개 (11.0MB)
tuned: json 11개 (4.1MB)
plain은 21개가 추적 밖 잔류

```
-- Explorer: 7회차 · grow snapshot·메타데이터 비교
-- table_kind | metric       | item_count
-- plain      | snapshots    | 120
-- plain      | metadata_log | 101        -- MinIO 실물 metadata.json: 122개 (11.0 MB)
-- tuned      | snapshots    | 120
-- tuned      | metadata_log | 11         -- MinIO 실물 metadata.json:  11개 ( 4.1 MB)
-- grow plain 데이터 파일 크기: data_files = 120, avg_bytes = 1227.8
```

실측(11/7) · 테이블 속성 두 줄만으로 metadata.json 객체 수가 122개(11.0MB)에서 11개(4.1MB)로 줄었습니다

compact: 합쳐도 공간이 줄지 않는 이유

rewrite_data_files 결과

rewritten_data_files: 120
added_data_files: 1
rewritten_bytes: 147,337

현재 vs 전체 데이터 파일

.files(현재 스냅샷): 1개
.all_data_files(고유): 121개
MinIO 데이터 객체: 121개

조회 주의 (.all_*)

스냅샷마다 같은 파일 반복
grow 직후 219 vs 실제 120
DISTINCT 경로로 집계

```
-- labs/session07/lab.py: compact 실행 및 파일 수 비교
CALL lake.system.rewrite_data_files(table => 'lesson07.s07_demo_compact_plain',
  options => map('min-input-files', '2'));

SELECT 'files' AS metric, count(*) AS cnt FROM lake.lesson07.s07_demo_compact_plain.files
UNION ALL
SELECT 'all_data_files_distinct', count(DISTINCT file_path)
FROM lake.lesson07.s07_demo_compact_plain.all_data_files;  -- 결과: 1 vs 121
```

실측(11/7) · 파일 120개를 1개로 합쳐도 이전 스냅샷 120개가 옛 파일을 참조해 저장소 객체는 121개로 늘었습니다

expire: 이제야 옛 파일이 지워진다

expire_snapshots 결과

deleted_data_files: 120
deleted_manifest_files: 120
deleted_manifest_lists: 120

데이터 객체 (121 → 1)

남은 스냅샷: 1개
.all_data_files(고유): 1개
데이터 용량: 988KB → 24~28KB

남은 숙제 두 가지

매니페스트: 현재·전체 22개
plain metadata.json: 124개
(tuned는 11개·1.4MB 유지)

```
-- labs/session07/lab.py: 스냅샷 만료 및 매니페스트 재작성
CALL lake.system.expire_snapshots(
  table => 'lesson07.s07_demo_expire_plain',
  older_than => TIMESTAMP '...', retain_last => 1);

CALL lake.system.rewrite_manifests('lesson07.s07_demo_clean_plain');
-- clean 후: manifests 1개 (all_manifests_distinct 23개, 다음 expire 때 정리)
```

실측(11/7) · expire_snapshots 뒤에야 옛 데이터 파일 120개가 지워지고, 매니페스트는 rewrite_manifests로 합칩니다

고아 파일과 24시간 안전장치

실험 테이블이 0개인 이유

remove_orphan_files는 기본 3일,
최소 24시간 이전 파일만 허용
진행 중 커밋의 파일 삭제 방지

공통 Bronze dry run 실측

older_than: 11/6 04:00 UTC
테이블당 고아 2,863~2,865개
metadata.json 2,852~2,854
+ avro 11

S3FileIO 환경 주의점

prefix_listing => true 필수
생략 시 Hadoop s3 스킴 오류
dry_run => true로 목록
선확인

실측(11/7) + 코드 · 추적 밖 metadata.json은 하루가 지난 뒤 remove_orphan_files로만 지울 수 있습니다

6회차 함정: 증분 시작점을 지우면

실험 설정 (inc 테이블)

1~3번 커밋 후 2번째를 progress
4~6번 추가 커밋 (끝 = main)
progress → end 증분 조회 실험

안전한 만료 (성공)

older_than = progress 커밋 시각
progress 스냅샷 보존됨
changelog 뷰 4행 정상 반환

위험한 만료 (실패)

older_than = 현재, retain_last = 1
최신 스냅샷 1개만 남음
not a parent ancestor 오류

```
-- labs/session07/lab.py: progress_trap 실험 (inc 테이블)
-- 1) 안전한 정리: progress 시각 기준 만료 → changelog 4행 성공
CALL lake.system.expire_snapshots(table => 'lesson07.s07_demo_trap_inc',
  older_than => TIMESTAMP '<progress_committed_at>', retain_last => 1);
-- 2) 위험한 정리: 현재 시각 기준 retain_last => 1 만료 후 changelog 호출
CALL lake.system.create_changelog_view(table => 'lesson07.s07_demo_trap_inc',
  options => map('start-snapshot-id', '<progress>', 'end-snapshot-id', '<end>'), ...);
-- 오류: Starting snapshot (exclusive) ... is not a parent ancestor of end snapshot ...
```

실측(11/7) · 하류 증분 작업이 기억한 progress 스냅샷을 지우면 create_changelog_view가 즉시 깨집니다

운영 기준: 순서·주기·지킬 것

항목	권장 순서 · 설정	지킬 안전선과 우리 Bronze 처방 (설계)
① 실행 순서	compact → expire → orphans → manifests	합친 뒤 만료해야 옛 데이터 파일이 실제로 삭제됨
② 스냅샷 보존	older_than < 하류 최소 progress	6회차 changelog 조상 · Flink 최신 커밋 스냅샷 보호
③ 메타데이터 속성	delete-after-commit=true, max=10~100	이후 커밋부터 자동 정리 · 이미 밀려난 파일은 orphans로 1회
④ 커밋 주기·감시	checkpoint·빈 커밋 간격 + 전후 측정	메타 테이블(.snapshots·.files)과 저장소 용량 함께 감시

설계 · 유지보수는 데이터 값(행 수·합계)을 바꾸지 않으면서 하류 progress와 Flink 복구 지점을 남겨야 합니다

프로젝트 리뷰 진행 방식

구간	할 일	초점
① 구조·흐름 설명	내 도메인·원천·Bronze~Gold 흐름 공유	누가 어떤 판단에 쓰는 데이터인가
② 설계·운영 문답	핵심 질문 6개 + 6·7회차 증분·유지보수 질문	정답 심사가 아니라 빈칸을 함께 채우는 시간
③ 한 줄 피드백	다음 주 8회차 연결 전에 보완할 우선순위 1~2개	수업 중 별도 제출 절차 없음 (추후 공지)

1인 발표·문답 시간은 오늘 참석 인원에 맞춰 똑같이 나누어 진행합니다

리뷰 질문지

점검 영역	내 프로젝트에 던질 질문
① 목적과 반영 주기	누가 어떤 판단에 쓰는 데이터인가? · 얼마나 빨리 반영돼야 하나?
② 변경과 정합성	수정·삭제·중복 전달은 어떻게 처리하나? · 맞다는 근거(테스트·대조)는 무엇인가?
③ 실패 복구와 선택 이유	실패하면 어디서 다시 시작하나? · 그 구조를 고른 이유와 감수한 비용·제약은?
④ 6·7회차 증분·유지보수	증분 범위와 영향 키 전파는? · 재시도 시 같은 답인가(멱등성)? · 커밋·스냅샷 정리 주기는?

핵심 질문 6개에 6·7회차의 증분·멱등성·유지보수 질문 3개를 더해 내 파이프라인을 점검합니다

리뷰 시간

① 설명 (절반 시간)

도메인 · 원천과 최종 마트
Bronze → Silver → Gold 흐름
가장 신경 쓴 설계 한 가지

② 질문 · 문답 (절반 시간)

수정·삭제·중복과 정합성 근거
증분 키 전파와 재시도 멱등성
커밋 주기 · 스냅샷 보존·정리

③ 한 줄 피드백

잘 잡힌 기준 한 가지 확인
8회차 전에 채울 빈칸 한 가지
다음 순서로 바로 전환

오늘 나온 질문과 피드백을 메모해 두었다가 8회차 운영 연결에 반영합니다

오늘 남길 것과 8회차 예고

유지보수 3원칙

compact 뒤 expire로 파일 삭제
하류 progress · Flink 스냅샷 보존
delete-after-commit으로 메타 관리

집에서 해 볼 실습

demo.sh와 Explorer 예제 8개
lab.sh로 단계별 전후 수치 확인
내 테이블 유지보수 기준표 작성

8회차 예고 (마무리)

Airflow DAG 연동과 재시도
유지보수 주기 태스크 연결
장애 복구와 최종 운영 점검

다음 시간: 6회차 증분과 7회차 유지보수를 Airflow 스케줄과 장애 복구 흐름으로 묶어 마무리합니다